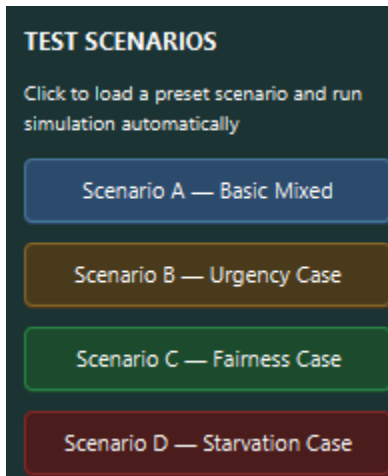


Round robin VS Priority Queue



Each button clears RQ then loads it with different processes to test:

1.Normal workload (Basic mixed):

Loads 5 processes with varied arrival times, burst times, and priorities.

Serves as the baseline comparison between Round Robin and Priority Scheduling.

Round Robin distributes CPU time evenly; Priority Scheduling favors lower priority numbers.

Expected: RR wins on fairness; PQ may win on waiting time for high-priority processes.

PROCESS TABLE				
PID	Arrival	Burst	Priority	
1	0	5	3	
2	1	3	1	
3	2	8	2	
4	3	2	4	
5	4	6	2	

```
public static SimulationResult loadScenarioA() { 1 usage HassanAhmed017
    Clear();
    quantum = 3;

    processes.add(new Process( pid: 1, arrivalTime: 0, burstTime: 5, priority: 3));
    processes.add(new Process( pid: 2, arrivalTime: 1, burstTime: 3, priority: 1));
    processes.add(new Process( pid: 3, arrivalTime: 2, burstTime: 8, priority: 2));
    processes.add(new Process( pid: 4, arrivalTime: 3, burstTime: 2, priority: 4));
    processes.add(new Process( pid: 5, arrivalTime: 4, burstTime: 6, priority: 2));

    for (Process p : processes) p.setResponseTime(-1);

    return start();
}
```

Result: RR win

① ROUND ROBIN

Gantt Chart



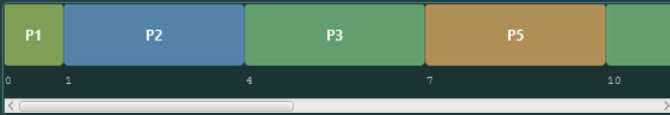
Results Table

PID	FT	TAT	WT	RT	
P1	5	5	0	0	
P2	8	7	4	4	
P3	24	22	14	6	
P4	13	10	8	8	
P5	22	18	12	9	
AVG	—	12.40	7.60	5.40	

Avg WT: 7.60 | Avg TAT: 12.40 | Avg RT: 5.40

② PRIORITY SCHEDULING

Gantt Chart



Results Table

PID	FT	TAT	WT	RT	
P1	22	22	17	0	
P2	4	3	0	0	
P3	18	16	8	2	
P4	24	21	19	19	
P5	16	12	6	3	
AVG	—	14.80	10.00	4.80	

Avg WT: 10.00 | Avg TAT: 14.80 | Avg RT: 4.80

③ COMPARISON SUMMARY

Metric	Round Robin	Priority
Avg Waiting Time	7.60	10.00
Avg Turnaround Time	12.40	14.80
Avg Response Time	5.40	4.80
Starvation Risk	None ✓	Possible ⚠
Fairness	High ✓	Low ✗
Best For	Interactive Systems	Batch / Priority Jobs

④ FINAL CONCLUSION

Q: Which algorithm had the shorter average waiting time?

A: Round Robin kept processes waiting less on average — 7.60 vs 10.00 units.

Q: Which algorithm completed processes faster overall?

A: Round Robin achieved a lower average turnaround time (12.40 vs 14.80 units), meaning processes spent less total time in the system.

Q: Which algorithm responded to processes more quickly?

A: Priority Scheduling gave processes their first CPU slot sooner on average (4.80 vs 5.40 units, a negligible difference).

Q: Did high-priority processes get a real advantage?

A: Yes — Priority Scheduling intentionally serves urgent processes first, so they finish much sooner than they would under Round Robin's fixed time-slice approach.

Q: Was Round Robin fair to all processes?

A: Yes — every process received CPU time in rotation, regardless of priority. No process was skipped or delayed because of a lower priority number.

Q: Could any process starve under Priority Scheduling?

A: Potentially yes — if high-priority processes keep arriving, a low-priority process may wait indefinitely. Round Robin avoids this by guaranteeing every process gets a turn.

Written Conclusion

CONCLUSION

Round Robin came out ahead in this simulation, winning 2 out of 3 measured metrics (waiting time, turnaround time, and response time).

Round Robin reduced average waiting time by 2.40 units compared to Priority Scheduling.

Round Robin finished processes 2.40 units sooner on average — a meaningful improvement in throughput for this workload.

From a fairness perspective, Round Robin guaranteed every process received CPU time regularly, making it the safer choice when no process should be left waiting indefinitely. Priority Scheduling, while faster for urgent tasks, introduces the risk of starvation for lower-priority processes if the workload is not carefully balanced.

2.Urgency case:

Loads 5 processes where P3 and P4 have priority=1 (highest urgency).

Tests whether Priority Scheduling visibly serves urgent processes before others.

Under PQ, P3 and P4 finish significantly earlier than under Round Robin.

Expected: PQ clearly outperforms RR in response time for high-priority processes.

PROCESS TABLE

PID	Arrival	Burst	Priority	
1	0	10	4	
2	1	8	4	
3	2	4	1	
4	3	6	1	
5	4	3	2	

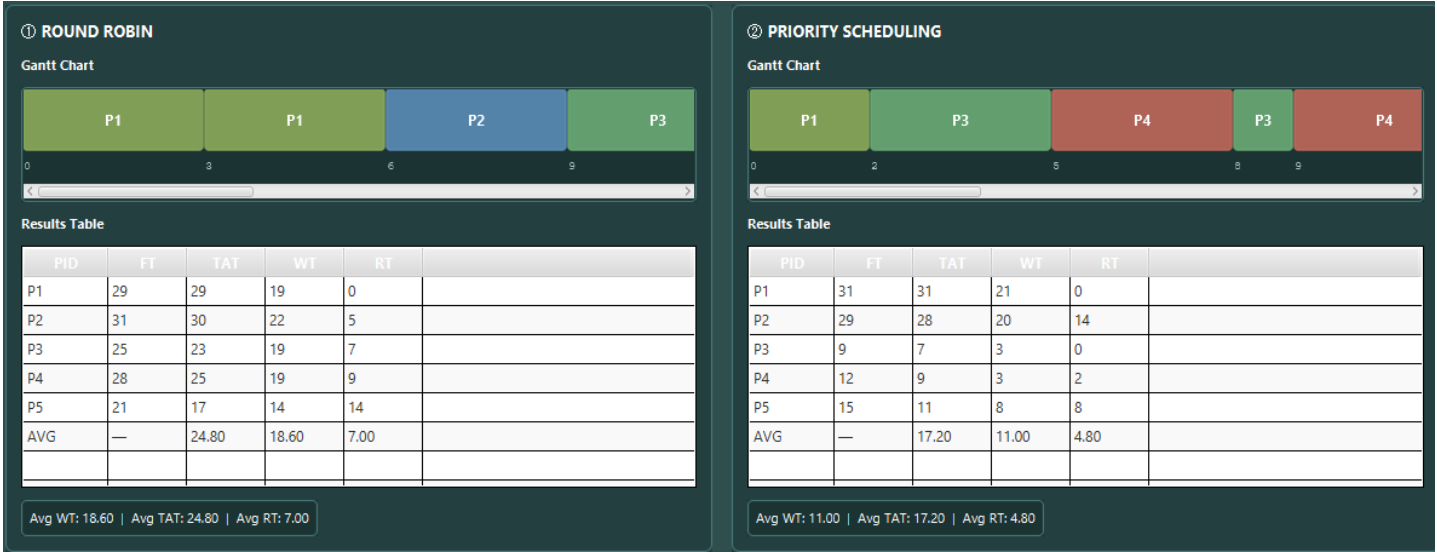
```
public static SimulationResult loadScenarioB() { 1 usage  HassanAhmed017 *
    Clear();|
    quantum = 3;

    processes.add(new Process( pid: 1, arrivalTime: 0, burstTime: 10, priority: 4));
    processes.add(new Process( pid: 2, arrivalTime: 1, burstTime: 8, priority: 4));
    processes.add(new Process( pid: 3, arrivalTime: 2, burstTime: 4, priority: 1));
    processes.add(new Process( pid: 4, arrivalTime: 3, burstTime: 6, priority: 1));
    processes.add(new Process( pid: 5, arrivalTime: 4, burstTime: 3, priority: 2));

    for (Process p : processes) p.setResponseTime(-1);

    return start();
}
```

Result: PQ win



③ COMPARISON SUMMARY		
Metric	Round Robin	Priority
Avg Waiting Time	18.60	11.00
Avg Turnaround Time	24.80	17.20
Avg Response Time	7.00	4.80
Starvation Risk	None ✓	Possible ⚠
Fairness	High ✓	Low ✗
Best For	Interactive Systems	Batch / Priority Jobs

④ FINAL CONCLUSION

Q: Which algorithm had the shorter average waiting time?

A: Priority Scheduling kept processes waiting less on average — 11.00 vs 18.60 units.

Q: Which algorithm completed processes faster overall?

A: Priority Scheduling achieved a lower average turnaround time (17.20 vs 24.80 units), meaning processes spent less total time in the system.

Q: Which algorithm responded to processes more quickly?

A: Priority Scheduling gave processes their first CPU slot sooner on average (4.80 vs 7.00 units).

Q: Did high-priority processes get a real advantage?

A: Yes — Priority Scheduling intentionally serves urgent processes first, so they finish much sooner than they would under Round Robin's fixed time-slice approach.

Q: Was Round Robin fair to all processes?

A: Yes — every process received CPU time in rotation, regardless of priority. No process was skipped or delayed because of a lower priority number.

Q: Could any process starve under Priority Scheduling?

A: Potentially yes — if high-priority processes keep arriving, a low-priority process may wait indefinitely. Round Robin avoids this by guaranteeing every process gets a turn.

Written Conclusion

CONCLUSION

Priority Scheduling came out ahead in this simulation, winning 3 out of 3 measured metrics (waiting time, turnaround time, and response time).

Priority Scheduling reduced average waiting time by 7.60 units compared to Round Robin.

Priority Scheduling finished processes 7.60 units sooner on average — a meaningful improvement in throughput for this workload.

From a fairness perspective, Round Robin guaranteed every process received CPU time regularly, making it the safer choice when no process should be left waiting indefinitely. Priority Scheduling, while faster for urgent tasks, introduces the risk of starvation for lower-priority processes if the workload is not carefully balanced.

3.Fairness case:

Loads 4 processes all arriving at $t=0$ with equal burst time (9) and equal priority (2).

Tests whether Round Robin distributes CPU time evenly across identical processes.

Under RR all processes receive equal time slices and finish at similar times; PQ is arbitrary.

Expected: RR shows perfect fairness; PQ finishes them sequentially with no fairness guarantee.

PROCESS TABLE

PID	Arrival	Burst	Priority	
1	0	9	2	
2	0	9	2	
3	0	9	2	
4	0	9	2	

```
public static SimulationResult loadScenarioC() { //is RR fair? 1 usage  HassanAhmed017
    Clear();
    quantum = 3;

    processes.add(new Process( pid: 1, arrivalTime: 0, brustTime: 9, priority: 2));
    processes.add(new Process( pid: 2, arrivalTime: 0, brustTime: 9, priority: 2));
    processes.add(new Process( pid: 3, arrivalTime: 0, brustTime: 9, priority: 2));
    processes.add(new Process( pid: 4, arrivalTime: 0, brustTime: 9, priority: 2));

    for (Process p : processes) p.setResponseTime(-1);

    return start();
}
```

Result: Tie

① ROUND ROBIN

Gantt Chart

P1

P2

P3

P4

0369

Results Table

PID	FT	TAT	WT	RT	
P1	27	27	18	0	
P2	30	30	21	3	
P3	33	33	24	6	
P4	36	36	27	9	
AVG	—	31.50	22.50	4.50	

Avg WT: 22.50 | Avg TAT: 31.50 | Avg RT: 4.50

② PRIORITY SCHEDULING

Gantt Chart

P1

P2

P3

P4

0369

Results Table

PID	FT	TAT	WT	RT	
P1	27	27	18	0	
P2	30	30	21	3	
P3	33	33	24	6	
P4	36	36	27	9	
AVG	—	31.50	22.50	4.50	

Avg WT: 22.50 | Avg TAT: 31.50 | Avg RT: 4.50

③ COMPARISON SUMMARY

Metric	Round Robin	Priority
Avg Waiting Time	22.50	22.50
Avg Turnaround Time	31.50	31.50
Avg Response Time	4.50	4.50
Starvation Risk	None ✓	Possible ⚠
Fairness	High ✓	Low ✗
Best For	Interactive Systems	Batch / Priority Jobs

④ FINAL CONCLUSION

Q: Which algorithm had the shorter average waiting time?

A: Round Robin kept processes waiting less on average — 22.50 vs 22.50 units, though the gap was very small.

Q: Which algorithm completed processes faster overall?

A: Round Robin achieved a lower average turnaround time (31.50 vs 31.50 units), meaning processes spent less total time in the system.

Q: Which algorithm responded to processes more quickly?

A: Round Robin gave processes their first CPU slot sooner on average (4.50 vs 4.50 units, a negligible difference).

Q: Did high-priority processes get a real advantage?

A: Yes — Priority Scheduling intentionally serves urgent processes first, so they finish much sooner than they would under Round Robin's fixed time-slice approach.

Q: Was Round Robin fair to all processes?

A: Yes — every process received CPU time in rotation, regardless of priority. No process was skipped or delayed because of a lower priority number.

Q: Could any process starve under Priority Scheduling?

A: Potentially yes — if high-priority processes keep arriving, a low-priority process may wait indefinitely. Round Robin avoids this by guaranteeing every process gets a turn.

Written Conclusion

CONCLUSION

Both algorithms performed equally well across the three measured metrics — this workload does not clearly favor either approach.

Waiting time was nearly identical between the two algorithms (22.50 for RR vs 22.50 for Priority), suggesting this workload does not strongly benefit from prioritization.

Turnaround time was also very close, meaning both algorithms completed this set of processes in roughly the same total time.

From a fairness perspective, Round Robin guaranteed every process received CPU time regularly, making it the safer choice when no process should be left waiting indefinitely. Priority Scheduling, while faster for urgent tasks, introduces the risk of starvation for lower-priority processes if the workload is not carefully balanced.

4.Starvation case:

Loads 4 high-priority processes (priority=1) and one low-priority process P5 (priority=4).

Tests whether a low-priority process suffers significantly longer waiting under Priority Scheduling.

Under PQ, P5 waits until all 4 higher-priority processes complete before receiving any CPU time.

Expected: P5 has near-zero wait under RR but maximum wait under PQ, demonstrating starvation risk.

PROCESS TABLE

PID	Arrival	Burst	Priority	
1	0	3	1	
2	0	3	1	
3	0	3	1	
4	0	3	1	
5	0	10	4	

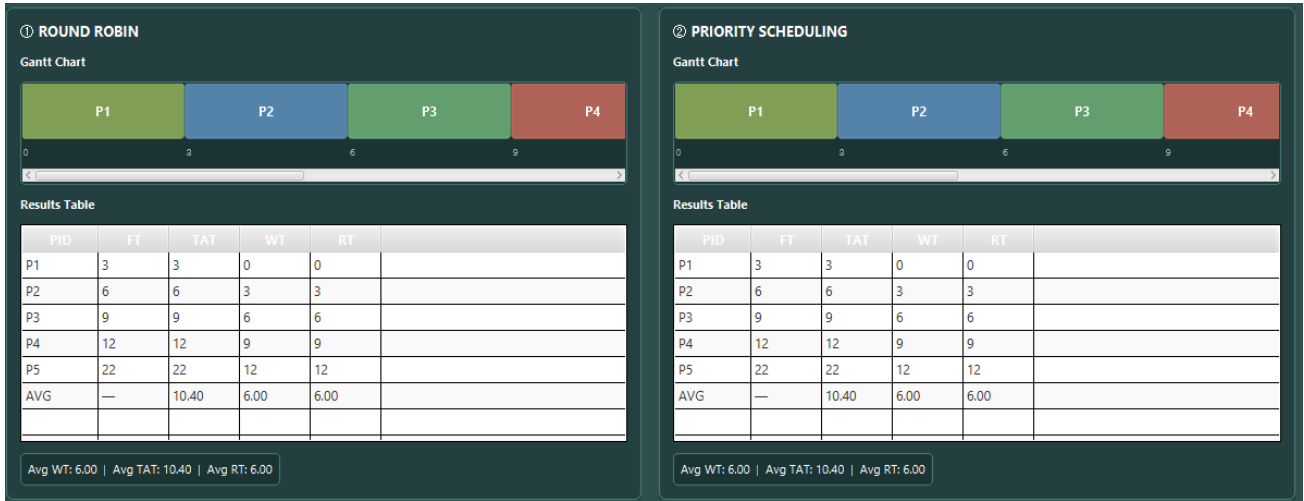
```
public static SimulationResult loadScenarioD() { //low priority waits more 1 usage HassanAhmed017
    Clear();
    quantum = 3;

    processes.add(new Process( pid: 1, arrivalTime: 0, brustTime: 3, priority: 1));
    processes.add(new Process( pid: 2, arrivalTime: 0, brustTime: 3, priority: 1));
    processes.add(new Process( pid: 3, arrivalTime: 0, brustTime: 3, priority: 1));
    processes.add(new Process( pid: 4, arrivalTime: 0, brustTime: 3, priority: 1));
    processes.add(new Process( pid: 5, arrivalTime: 0, brustTime: 10, priority: 4));

    for (Process p : processes) p.setResponseTime(-1);

    return start();
}
```

Result: Tie



③ COMPARISON SUMMARY		
Metric	Round Robin	Priority
Avg Waiting Time	6.00	6.00
Avg Turnaround Time	10.40	10.40
Avg Response Time	6.00	6.00
Starvation Risk	None ✓	Possible ⚠
Fairness	High ✓	Low ✗
Best For	Interactive Systems	Batch / Priority Jobs

④ FINAL CONCLUSION

Q: Which algorithm had the shorter average waiting time?

A: Round Robin kept processes waiting less on average — 6.00 vs 6.00 units, though the gap was very small.

Q: Which algorithm completed processes faster overall?

A: Round Robin achieved a lower average turnaround time (10.40 vs 10.40 units), meaning processes spent less total time in the system.

Q: Which algorithm responded to processes more quickly?

A: Round Robin gave processes their first CPU slot sooner on average (6.00 vs 6.00 units, a negligible difference).

Q: Did high-priority processes get a real advantage?

A: Yes — Priority Scheduling intentionally serves urgent processes first, so they finish much sooner than they would under Round Robin's fixed time-slice approach.

Q: Was Round Robin fair to all processes?

A: Yes — every process received CPU time in rotation, regardless of priority. No process was skipped or delayed because of a lower priority number.

Q: Could any process starve under Priority Scheduling?

A: Potentially yes — if high-priority processes keep arriving, a low-priority process may wait indefinitely. Round Robin avoids this by guaranteeing every process gets a turn.

Written Conclusion

CONCLUSION

Both algorithms performed equally well across the three measured metrics — this workload does not clearly favor either approach.

Waiting time was nearly identical between the two algorithms (6.00 for RR vs 6.00 for Priority), suggesting this workload does not strongly benefit from prioritization.

Turnaround time was also very close, meaning both algorithms completed this set of processes in roughly the same total time.

From a fairness perspective, Round Robin guaranteed every process received CPU time regularly, making it the safer choice when no process should be left waiting indefinitely. Priority Scheduling, while faster for urgent tasks, introduces the risk of starvation for lower-priority processes if the workload is not carefully balanced.

For more Details and code: [Git-Hub repository](#)